Modular

LLVM DEVELOPERS' MEETING 2026

Beyond record and replay

Replacing record & replay in the MAX graph compiler

Spenser Bauman | Modular

Part 1

Device graphs: what and why

launch overhead, and what a graph is

1 **Device graphs: what and why**

2 Record & replay today

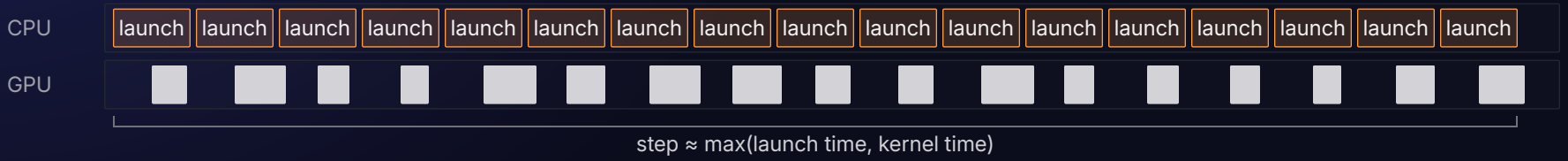
3 Device graph synthesis

4 Multi-device models

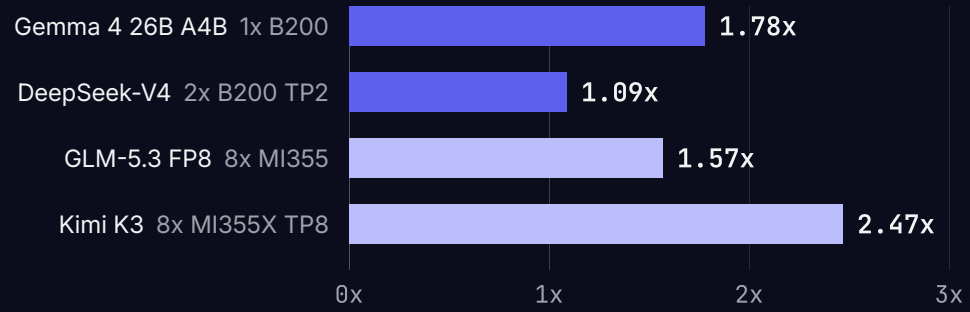
5 EmitMojo backend

The GPU is waiting on the CPU

Eager: one launch call per kernel



Device graph: one launch for the whole step

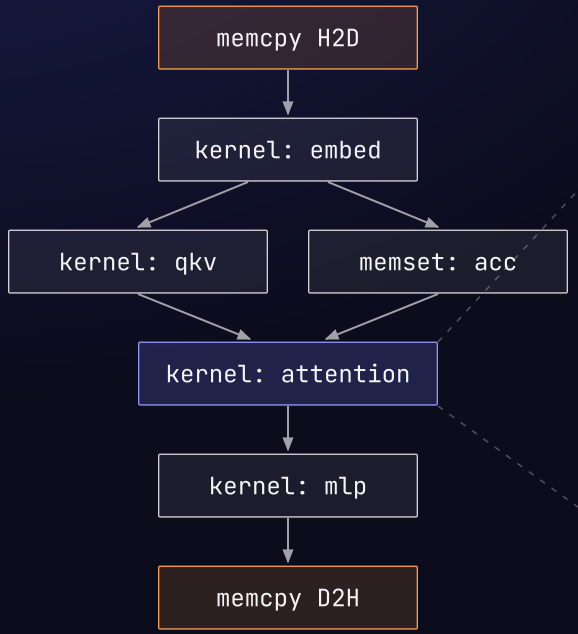


CUDA HIP

Speedup in ms per output token, eager ÷ replayed.

What's in a device graph

A DAG of device work. Each node carries everything needed to launch it.



nodes: kernel, memcpy, memset, empty
edges: dependencies

Attention node

function	`attention_kernel[bf16, head_dim=128]`
grid / block	`(64, 1, 1) / (256, 1, 1)`
shared memory	`48 KB`
arguments	`q = 0x7f3a4000, k = 0x7f3a8000, v = 0x7f3ac000, out = 0x7f3b0000, seq_len = 2048`
depends on	`kernel: qkv, memset: acc`

Every field is a value, fixed when the node is added: pointers, extents, launch shape.

Part 2

Record & replay today

what it is and why we don't like it

1 Device graphs: what and why

2 **Record & replay today**

3 Device graph synthesis

4 Multi-device models

5 EmitMojo backend

Record and replay with CUDA and HIP graphs

CUDA

```
cudaGraph_t graph;
cudaGraphExec_t exec;

// record: launches on a capturing stream become graph nodes
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
embed<<<grid, block, 0, stream>>>(tokens, x);
attention<<<grid, block, 0, stream>>>(x, kv_cache, y);
mlp<<<grid, block, 0, stream>>>(y, out);
cudaStreamEndCapture(stream, &graph);

// instantiate once
cudaGraphInstantiate(&exec, graph, 0);

// replay: one call per step
for (int step = 0; step < num_steps; ++step) {
  update_inputs(tokens);  // same addresses, new data
  cudaGraphLaunch(exec, stream);
}
```

1 Record

Kernels launched on a capturing stream are added to a graph instead of running.

2 Instantiate

The graph is validated and turned into an executable graph, once.

3 Replay

One launch call runs every recorded node, with the recorded arguments.

HIP mirrors the API one to one

CUDA	HIP
`cudaStreamBeginCapture`	`hipStreamBeginCapture`
`cudaStreamEndCapture`	`hipStreamEndCapture`
`cudaGraphInstantiate`	`hipGraphInstantiate`
`cudaGraphLaunch`	`hipGraphLaunch`

Capture and replay in MAX

Warmup: one captured graph per (batch size, cache-length bucket)

```
key = pack_graph_key(batch_size, cache_len)
# buffers the graph keeps for its whole life
inputs = allocate_inputs(batch_size, cache_len)
# eager run under stream capture
outputs = model.capture(key, *inputs)
graph_entries[key] = (inputs, outputs)
```

Every decode step: replay

```
key = pack_graph_key(len(batch), round_up(cache_len, cache_length_buckets))
captured_inputs, outputs = graph_entries[key]
# same addresses, new data
batch_inplace_copy(captured_inputs, live_inputs)
# one graph launch per device; outputs are written in place
model.replay(key, *captured_inputs)
return outputs
```

`capture(key, *inputs)`

Runs the model once under stream capture and returns the output buffers the graph writes.

`replay(key, *inputs)`

Same buffers, same addresses. Copy new data in first; one graph launch per device follows.

The contract

Inputs keep their addresses for the life of the graph, and nothing on the host runs during replay.

What capture cannot give us

HARD OR IMPOSSIBLE WITH CAPTURE

Targets without a capture mode

impossible

Several accelerators expose graph or command-buffer construction but no record/replay: Qualcomm, Metal, Trainium

Control flow inside a graph

hard

Capture records one trace. A conditional node has to be constructed, and threading a branch through a stream capture of an async runtime is messy.

More than one graph per model

impossible

The only capture boundary it can offer is the whole model.

Centralize device graph handling

hard

Record & replay spreads significant complexity across the stack: graph executor, async runtime, memory pools, kernels, and driver implementations.

WHAT WE WANT

One source of truth: the model definition

Creation

The device graph's structure is derived from the model definition (IR).

Caching

The key is derived from the model's interface: inputs, shapes, host values.

Execution

Input/output handling is derived from the model's signature.

Today these live in Python warmup, engine key hashing, and the driver.

Part 3

Device graph synthesis

the graph compiler, IR, and builder APIs

1 Device graphs: what and why

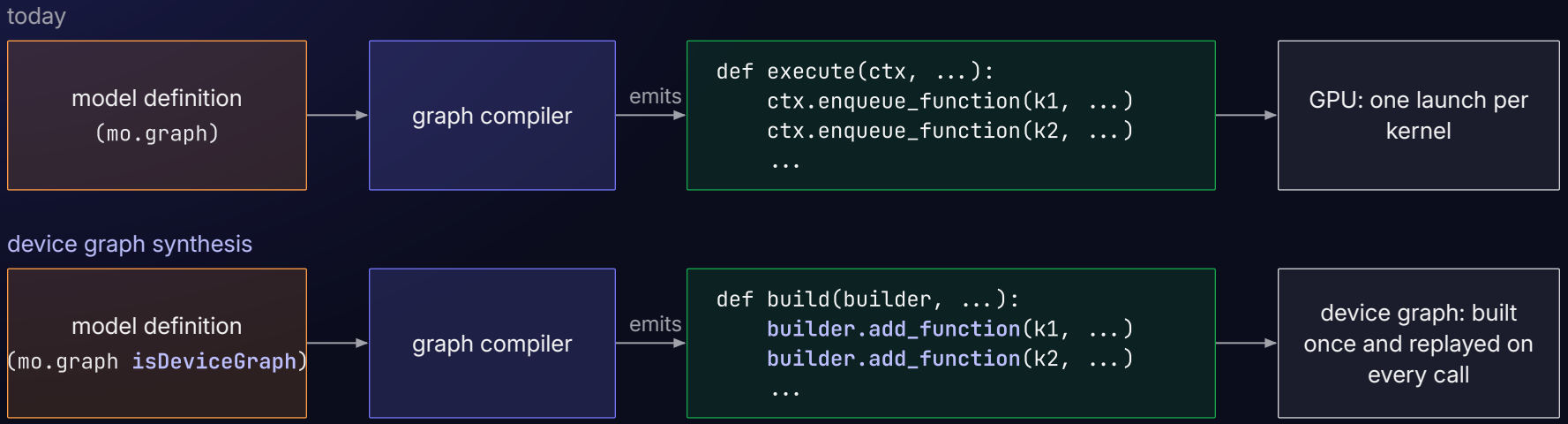
2 Record & replay today

3 **Device graph synthesis**

4 Multi-device models

5 EmitMojo backend

The idea: the compiler emits graph-building code



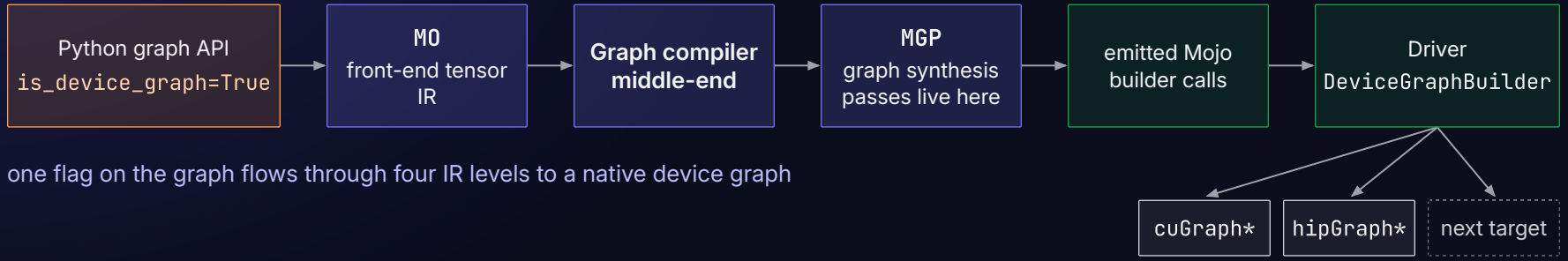
same compiler, same kernels: the emitted Mojo adds nodes to a graph instead of launching them

---

Co-design

The Mojo builder API was designed first, as something a kernel author would use by hand. The IR ops were then shaped to lower onto it one to one, so the IR stays thin and the Mojo library carries the semantics.

End to end: opt-in with a single flag



one flag on the graph flows through four IR levels to a native device graph

```
with Graph(
    "mlp_layer",
    input_types=[x_type, w_type],
    is_device_graph=True,
) as graph:
    x, w = (v.tensor for v in graph.inputs)
    graph.output(ops.gelu(x @ w))
# loading and calling the model are unchanged
model = session.load(graph)
```

MO

Front-end tensor graph. What the Python graph API builds. A device graph is a mo.graph with one attribute.

MGP

Imperative execution IR: buffers, devices, async values. The synthesis passes live here, and Mojo is emitted from it.

The builder API in Mojo

By hand: explicit edges between nodes

```
var ctx = DeviceContext("gpu:0")
var acc = ctx.enqueue_create_buffer[.float32](n)
var out = ctx.enqueue_create_host_buffer[.float32](n)

def build(mut b: DeviceGraphBuilder) raises {imm}:
    # every node names its predecessors; none means a root
    var zero = b.add_memset(acc, Float32(0))
    var k0 = b.add_function[scale_kernel](
        x, acc, grid_dim=blocks, block_dim=256,
        dependencies=[zero])
    var k1 = b.add_function[reduce_kernel](
        acc, grid_dim=1, block_dim=256,
        dependencies=[k0])
    # device to host
    _ = b.add_copy(out, acc, dependencies=[k1])

# runs `build` once, compiles, instantiates
var graph = DeviceGraph.create(ctx, build)
graph.replay()
graph.replay()  # as many times as needed
```

Emitted form: regions chain nodes

```
def work(mut b: DeviceGraphBuilder) raises {imm}:
    # nodes chain in program order
    b.add_function[scale_kernel](
        x, acc, grid_dim=blocks, block_dim=256)
    b.add_function[reduce_kernel](
        acc, grid_dim=1, block_dim=256)

# one handle for everything the region recorded
var node = b.region(work, dependencies=[zero])
_ = b.add_copy(out, acc, dependencies=[node])
```

Backends behind one C++ builder interface

CUDA	**HIP**	**Qualcomm**
cuGraph*	hipGraph*	prototype

An end-to-end example

```
mo.graph @mlp_layer(%x: !mo.tensor<[4096, 4096], bf16, gpu:0>,
                    %w: !mo.tensor<[4096, 4096], bf16, gpu:0>)
    -> !mo.tensor<[B, 4096], bf16, gpu:0>
    attributes {isDeviceGraph} {
  %h = mo.matmul(%x, %w) : (!mo.tensor<[4096, 4096], bf16, gpu:0>, !mo.tensor<[4096, 4096], bf16, gpu:0>)
      -> !mo.tensor<[4096, 4096], bf16, gpu:0>
  %y = mo.gelu(%h) : !mo.tensor<[4096, 4096], bf16, gpu:0>
  mo.output %y : !mo.tensor<[4096, 4096], bf16, gpu:0>
}
```

Only validation at this level

Verify the absence of unsupported features like control flow.

The mo.graph the source of truth

The graph's signature is its interface. Inputs, outputs, and the cache key are derived from it.

The synthesis pipeline

MGP, after primitive lowering

then the ordinary emit pipeline

Partition → **Compute Dependencies** → **Finalize**

one region per device

dataflow becomes explicit node edges, then simplified

inputs, cache, create + execute ops

every pass is a no-op unless the model carries `isDeviceGraph`

What flows in: the execute block of an `isDeviceGraph` model, every kernel call already compiled into a region

```
mgp.model @mlp_layer ... execute (
    %gpu: !mgp.device_context<<"gpu", 0>>,
    %x: !mgp.tensor<4096x4096xbf16, <"gpu", 0>>,
    %w: !mgp.tensor<4096x4096xbf16, <"gpu", 0>>) {
  %hb = mogg.routine.async (%gpu) { ... }              // alloc
  %h = mogg.routine.async (%x, %w, %hb, %gpu) { ... }  // matmul
  %yb = mogg.routine.async (%gpu) { ... }              // alloc
  %y = mogg.routine.async (%h, %yb, %gpu) { ... }      // gelu
  mef.output %y
} {isDeviceGraph}
```

Partition: everything becomes graph-building code

1 Lower to compiled regions

Every remaining op becomes a `mogg.routine.async`, the unit the kernel path already compiles.

2 Wrap in a collective

The body moves into `mgp.device_graph.collective` with an explicit `DeviceGraphBuilder`, one region per device.

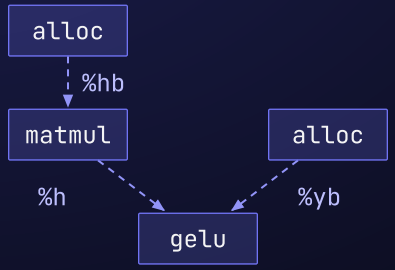
3 Context becomes builder

Every use of the device context in the body is replaced by the builder: a launch is now a node.

The `@mlp_layer` example after these passes. The regions took `%gpu` before; they take `%b` now.

```
%y = mgp.device_graph.collective builder(%b: !mgp.device_graph_builder) <"gpu", 0> {
  %hb = mogg.routine.async (%b) { ... }                // alloc
  %h  = mogg.routine.async (%x, %w, %hb, %b) { ... }   // matmul
  %yb = mogg.routine.async (%b) { ... }                // alloc
  %y0 = mogg.routine.async (%h, %yb, %b) { ... }       // gelu
  mgp.device_graph.collective.yield %y0
}
```

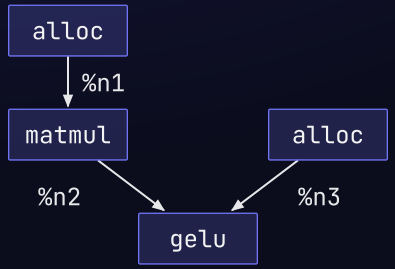
Compute Dependencies: derive execution order from dataflow



BEFORE

order implied by SSA uses

order is inferred from dataflow



AFTER

edges are node handles

compute deps + simplify

After this pass the edges are the order. Simplification merges chains and drops redundant edges, which shrinks the emitted Mojo rather than the graph.

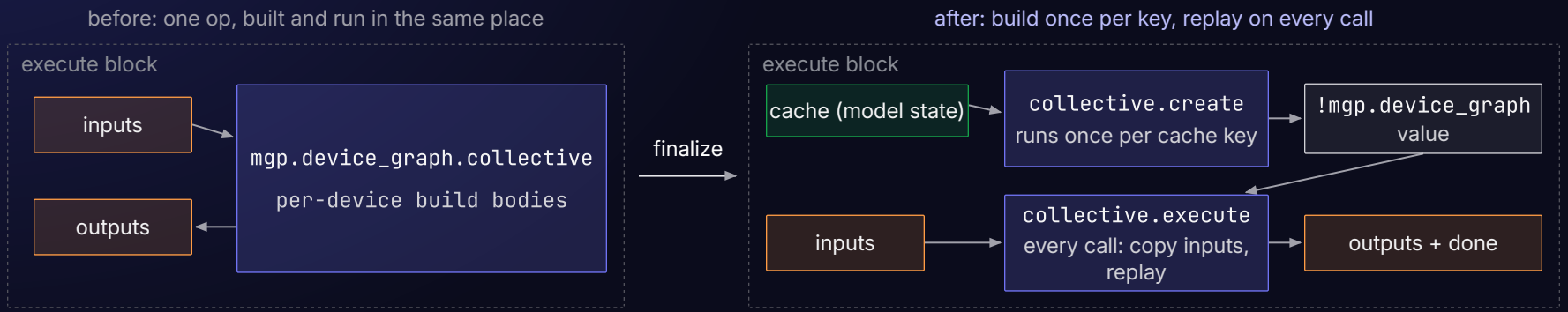
before

```
mgp.device_graph.collective builder(%b) <"gpu", 0> {
  %hb = mogg.routine.async (%b) { ... }
  %h = mogg.routine.async (%x, %w, %hb, %b) { ... }
  %yb = mogg.routine.async (%b) { ... }
  %y0 = mogg.routine.async (%h, %yb, %b) { ... }
  mgp.device_graph.collective.yield %y0
}
```

after

```
%n1, %hb = mgp.device_graph.dependencies
    builder(%b) dependencies() { ... }
%n2, %h = mgp.device_graph.dependencies
    builder(%b) dependencies(%n1) { ... }
%n3, %yb = mgp.device_graph.dependencies
    builder(%b) dependencies() { ... }
%n4, %y0 = mgp.device_graph.dependencies
    builder(%b) dependencies(%n2, %n3) { ... }
mgp.device_graph.collective.yield %y0
```

Finalize: build once per key, replay on every call



the graph's signature is derived from the values the body uses: device inputs, shape-only inputs, no chains

```
%cache = mgp.device_graph.cache %ctx : !mgp.device_graph.cache
%graph = mgp.device_graph.collective.create %ctx cache(%cache) graph(%gpu) builder(%b0)
    inputs(%x : !t) {
  ...  // the per-device build bodies, unchanged
  mgp.device_graph.yield %y : !t
} : !mgp.device_graph<(!t) -> !t>
%done, %y = mgp.device_graph.collective.execute %graph inputs(%x : !t)  // replay
    : !mgp.device_graph<(!t) -> !t> -> !t
mef.output %done, %y : !mef.chain, !t
```

Part 4

Multi-device models

partitioning multi-device computation

1 Device graphs: what and why

2 Record & replay today

3 Device graph synthesis

4 **Multi-device models**

5 EmitMojo backend

Goal: one device graph per-device

each device's allreduce reads the other device's partial: a buffer another device's graph owns

```
!t0 = !mo.tensor<[B, 4096], bf16, gpu:0>
!t1 = !mo.tensor<[B, 4096], bf16, gpu:1>
mo.graph @tp_mlp(%x0: !t0, %x1: !t1, %sig0, %sig1)
    → (!t0, !t1) attributes {isDeviceGraph} {
  %p0 = mo.matmul(mo.gelu(mo.matmul(%x0, %w1_0)), %w2_0)  // gpu:0 shard
  %p1 = mo.matmul(mo.gelu(mo.matmul(%x1, %w1_1)), %w2_1)  // gpu:1 shard
  // one allreduce per device; each reads both partials
  %y0 = mo.bundled.allreduce.sum(%p0, %p1, %sig0, %sig1) : (...) → !t0
  %y1 = mo.bundled.allreduce.sum(%p0, %p1, %sig0, %sig1) : (...) → !t1
  mo.output %y0, %y1 : !t0, !t1
}
```

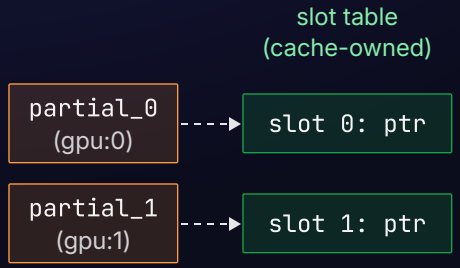
Cross-device communication

- gpu:0 needs partial_0's buffer address, and
- gpu:1 needs partial_1's buffer address, but
- partial_0 and partial_1 are allocated by separate device graph regions

Stable slots for cross-device tensors

Stable slot: a pointer the cache owns for the life of the collective. Producers publish into it before each replay; consumers' graphs read through the slot.

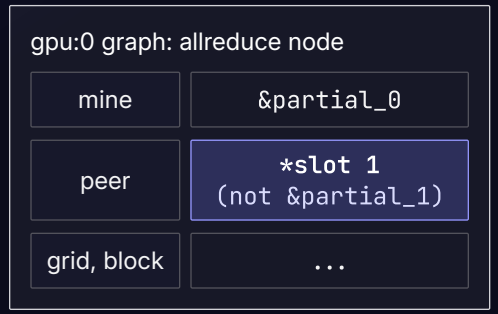
1 assign a slot per cross-device tensor



the slot's own address never moves; the tensor's may

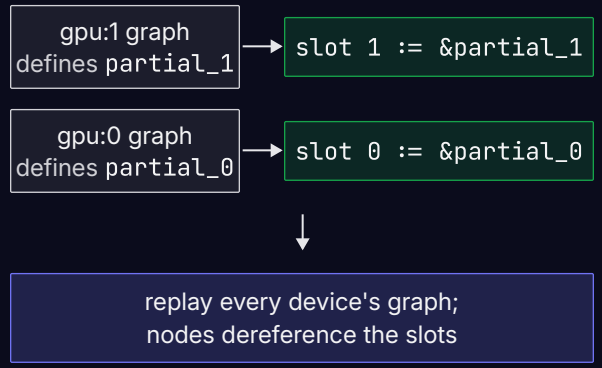
a slot is a device pointer to a device pointer

2 bake the slot into the graph



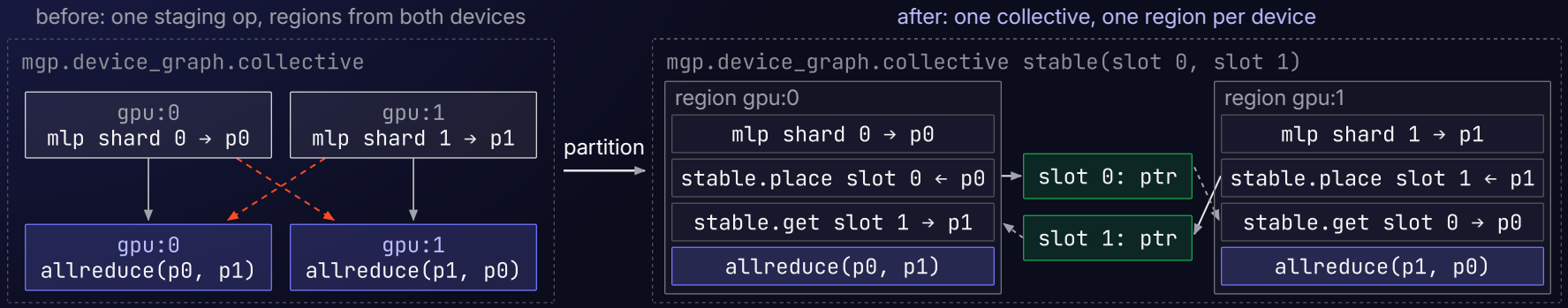
the node reads the peer through the slot, so a rebuilt peer graph never invalidates it

3 publish, then replay



before every execution, each graph publishes the tensors it defines into their slots

Partitioning: one region per device




```
mgp.device_graph.collective builder(%b)
    stable(!mgp.tensor<Bx4096xbf16, <"gpu", 0>>>,!mgp.tensor<Bx4096xbf16, <"gpu", 1>>>)
<"gpu", 0> {
  %p0 = mogg.routine.async (%x0, ...) { ... }          // mlp shard 0
  %t0 = mgp.device_graph.stable.place %b[0], %p0       // publish my partial
  %t1 = mgp.device_graph.stable.get %b[1]              // the peer's slot
  %y0 = mogg.routine.async (%t0, %t1, %sig0) { ... }   // allreduce
  mgp.device_graph.collective.yield %y0
}, <"gpu", 1> {
  ...                                                  // mirror: place %b[1], get %b[0]
}
```

Part 5

EmitMojo backend

the Mojo code produced by the graph compiler

1 Device graphs: what and why

2 Record & replay today

3 Device graph synthesis

4 Multi-device models

5 **EmitMojo backend**

The graph compiler emits Mojo code

```
def build_for[device: Int](mut builder: DeviceGraphBuilder) raises {imm}:
    comptime if device == 0:
        var x_stable = builder.add_input(x0)  # graph-owned input buffer
        var p0 = Optional[...](None)
        def work0(mut builder: DeviceGraphBuilder) raises {mut}:
            p0 = mgp_device_graph_alloc(spec, builder)
            stub_mlp_shard_build_graph(x_stable, p0.value(), builder)

        var n0 = builder.region(work0, dependencies = [])
        def work1(mut builder: DeviceGraphBuilder) raises {imm}:
            builder.place_stable(0, p0.value())   # publish my partial
            var p1 = builder.get_stable(1)        # read the peer's slot
            stub_allreduce_build_graph(p0.value(), p1, sig0, y0, builder)

        var n1 = builder.region(work1, dependencies = [n0])
        builder.add_output(y0.async_pack())
    elif device == 1:
        ...  # mirror image

var graphs = DeviceGraph.create_collective(ctxs, build_for)
```

Built once per key

The key closure is generated from the graph signature. Replay runs no generated code.

Regions are closures

Each dependencies op is one work closure

Slots are two calls

`place_stable` publishes this rank's partial; `get_stable` reads the peer's slot.

Replay: what one call does

```
def replay_collective[N: Int](
    graphs: Array[DeviceGraph, N], inputs: Array[List[Buffer], N]
) raises:
    # 1. publish stable slots: slots now point at this call's buffers
    for graph in graphs:
        graph.patch_stable_addresses()

    # 2. refresh inputs: copy live data into the graph's own input buffers
    #    (in-place inputs such as the KV cache are skipped)
    for graph, live_inputs in zip(graphs, inputs):
        for graph_input, live_input in zip(graph.graph_inputs, live_inputs):
            if not graph_input.in_place:
                live_input.copy_to(graph_input)

    # 3. replay: one graph launch per device
    for graph in graphs:
        graph.replay()

    # 4. the outputs are the buffers the graph recorded
    return [graph.outputs for graph in graphs]
```

Stable slots

Producers' buffer addresses are published before anything launches.

Graph-owned inputs

Replay never depends on the caller's addresses; live data is copied into the graph's own input buffers.

Nothing generated runs

The build closures ran once per key. Replay is a fixed runtime routine plus one graph launch per device.

Current status

WORKING TODAY

- a production Gemma 4 model runs end to end through synthesis on 1 or more GPUs
- multiple device-graph regions in one model (including control flow)
- direct graph construction ~40% faster than recording
- DeviceGraphBuilder implementations for: CUDA, HIP, AI100

OPEN

- control flow inside graphs, via conditional nodes
- more DeviceGraphBuilder implemententions

Takeaways

1. Launch overhead made device graphs mandatory.
2. Capture works, but it is an end-to-end concern with a stack-wide complexity cost.
3. Constructing the graph in the compiler makes it a first-class IR object with one interface, targeting vendor-neutral Mojo API.

Questions